MODULE *IntegrationContract*

Section: Imports

Shared operators used by this module are imported below.

EXTENDS *Naturals*, *FiniteSets*, *FailurePolicy*

Section: Integration boundary

This layer receives completed per-check snapshots from *Overwatch*. It specifies requests, responses, delivery channels, and session lifecycle for every host. Both layers use the same *FailurePolicy* definitions.

CONSTANTS *Hosts*, *Checks*, *Names*, *MaxRequests*, *MaxChanges*, *MaxSessions*, *MaxNotices*

AllHosts $\triangleq$ {"claude", "codex", "opencode1", "opencode2", "pi", "neovim"}
ASSUME *Hosts* $\subseteq$ *AllHosts* $\land$ *Hosts* $\neq \{\}$

These are host capabilities, not advisory/strict policy modes. A warning cannot be counted as a denied completion, and an editor has no agent gate.

Section: Host capabilities

The host determines whether protected requests wait for a running check, how guarded completion is handled, and which delivery channels exist. A host notices another process's change when it can tell it from the session's own: *Claude* Code reads each watcher result against the session's edits, *Codex* and *OpenCode* 2 see a change first at a hook that follows no edit of the session's, and an editor treats every save alike. Pi and *OpenCode* 1 check only after the session's own edits and at settlement, so they read every result as the session's. These are integration capabilities, not user-selectable enforcement modes.

```
Capabilities(h) ≜
    [agent ↦ h ≠ "neovim",
     waitsForCheck ↦ h ∈ {"codex", "opencode1", "opencode2", "pi"},
     completion ↦ CASE h ∈ {"claude", "codex"} → "block"
                    □  h = "opencode1" → "warn"
                    □  h = "opencode2" → "queue"
                    □  h = "pi" → "continue"
                    □  OTHER → "none",
     delivery   ↦ CASE h = "claude" → "turn"
                    □  h = "opencode2" → "context"
                    □  h = "neovim" → "quickfix"
                    □  OTHER → "event",
     idleWake ↦ h ∈ {"claude", "opencode2"},
     status ↦ h ∈ {"claude", "pi", "neovim"},
     noticesOutside ↦ h ∈ {"claude", "codex", "opencode2", "neovim"},
     tcr ↦ h ∈ {"codex", "opencode1", "opencode2", "pi"}]

Kinds   ≜ {"inspect", "repair", "publish", "complete"}
Empty   ≜ [c ∈ Checks ↦ {}]
Snapshots ≜ [Checks → SUBSET Names]
NoReply ≜ [ready ↦ FALSE, decision ↦ "none"]
```

Section: State shape

Request and reply are separate so that a check can finish or a session can close between them. Report visibility, deferred context, corrections, and idle wakeups are separate observations.

```
VARIABLES host, active, busy, reports, baseline, inherited, selected,
          request, reply, requests, changes, session,
          visible, contextPending, contextSent, correctionQueued,
          contexts, warnings, corrections, inTurn, wakes, work,
          correctionPending, correctionTurn, outside, outsides,
          noticePending, noticesRaised, noticesTold

vars ≜ ⟨host, active, busy, reports, baseline, inherited, selected,
        request, reply, requests, changes, session, visible,
        contextPending, contextSent, correctionQueued, contexts,
        warnings, corrections, inTurn, wakes, work,
```

$correctionPending,\ correctionTurn,\ outside,\ outsides,$
$noticePending,\ noticesRaised,\ noticesTold\rangle$
$noticeVars \triangleq \langle noticePending,\ noticesRaised,\ noticesTold\rangle$

Section: *Guard* selection and inherited policy

An empty selection means all checks. *FailureGuards* decides whether each selected check guards publication; *Notify* identifies new failures for delivery.

$Guarded(scope) \triangleq$
$\{c \in scope : FailureGuards(reports[c],\ baseline[c],\ inherited)\}$
$Guard \triangleq Capabilities(host).agent \land Guarded(Checks) \neq \{\}$
$Notify \triangleq Capabilities(host).agent \land$
$(\exists\, c \in Checks : FailureGuards(reports[c],\ baseline[c],\ \text{“allow”}))$
$SelectedChecks \triangleq \text{IF } selected = \{\} \text{ THEN } Checks \text{ ELSE } selected$

Section: State domains

Counters and snapshots are bounded for *TLC* exploration. These bounds do not assert a runtime limit on requests or sessions.

$TypeOK \triangleq$
$\land\ host \in Hosts \land active \in \text{BOOLEAN} \land busy \in \text{BOOLEAN}$
$\land\ work \in 0\,..\,((MaxSessions + 1) * MaxRequests)$
$\land\ inTurn \in \text{BOOLEAN}$
$\land\ wakes \in 0\,..\,(MaxChanges + MaxSessions + 1 + (MaxSessions + 1) * MaxRequests)$
$\land\ reports \in Snapshots \land baseline \in Snapshots$
$\land\ inherited \in \{\text{“allow”},\ \text{“block”}\} \land selected \subseteq Checks$
$\land\ request \in Kinds \cup \{\text{“none”}\}$
$\land\ reply \in [ready : \text{BOOLEAN},$
$decision : \{\text{“none”},\ \text{“allow”},\ \text{“deny”},\ \text{“block”},\ \text{“warn”},\ \text{“queue”},\ \text{“continue”},\ \text{“cancelled”}\}]$
$\land\ requests \in 0\,..\,MaxRequests \land changes \in 0\,..\,MaxChanges \land session \in 0\,..\,MaxSessions$
$\land\ visible \in [known : \text{BOOLEAN},\ failures : Snapshots]$
$\land\ contextPending \in \text{BOOLEAN} \land contextSent \in \text{BOOLEAN} \land correctionQueued \in \text{BOOLEAN}$
$\land\ contexts \in 0\,..\,(MaxChanges + MaxSessions + 1)$
$\land\ warnings \in 0\,..\,((MaxSessions + 1) * MaxRequests)$
$\land\ corrections \in 0\,..\,((MaxSessions + 1) * MaxRequests)$
$\land\ correctionPending \in \text{BOOLEAN} \land correctionTurn \in \text{BOOLEAN} \land outside \in \text{BOOLEAN}$
$\land\ outsides \in 0\,..\,MaxChanges$
$\land\ noticePending \in \text{BOOLEAN}$
$\land\ noticesRaised \in 0\,..\,MaxNotices \land noticesTold \in 0\,..\,MaxNotices$

Section: Initial state

A host starts active with arbitrary reports, baseline, policy, and check state. No reply, visible report, deferred context, or correction has been produced.

$Init \triangleq$
$\land\ host \in Hosts \land active = \text{TRUE} \land busy \in \text{BOOLEAN}$
$\land\ reports \in Snapshots \land baseline \in Snapshots$
$\land\ inherited \in \{\text{“allow”},\ \text{“block”}\} \land selected \in \text{SUBSET } Checks$
$\land\ request = \text{“none”} \land reply = NoReply$
$\land\ requests = 0 \land changes = 0 \land session = 0$
$\land\ visible = [known \mapsto \text{FALSE},\ failures \mapsto Empty]$
$\land\ contextPending = \text{FALSE} \land contextSent = \text{FALSE} \land correctionQueued = \text{FALSE}$
$\land\ contexts = 0 \land warnings = 0 \land corrections = 0 \land wakes = 0 \land work = 0 \land inTurn \in \text{BOOLEAN}$
$\land\ correctionPending = \text{FALSE} \land correctionTurn = \text{FALSE} \land outside = \text{FALSE} \land outsides = 0$
$\land\ noticePending = \text{FALSE} \land noticesRaised = 0 \land noticesTold = 0$

Section: Requests and completion decisions

Inspection and repair remain answerable during checks. Protected requests follow the host waiting capability and consult the selected reports. Respond records the decision and accounts separately for warnings, corrections, and wakeups; *Acknowledge* releases the request slot.

```
Request(k) ≜
    ∧  active ∧ request = "none" ∧ requests < MaxRequests
    ∧  request′ = k ∧ reply′ = NoReply ∧ requests′ = requests + 1
    ∧  UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                  changes, session, visible, contextPending, contextSent,
                  correctionQueued, contexts, warnings, corrections, inTurn, wakes, work,
                  correctionPending, correctionTurn, outside, outsides, noticeVars⟩

CanRespond ≜
    active ∧ request ≠ "none" ∧ ¬reply.ready ∧
    (request ∈ {"inspect", "repair"}      ∨ ¬Capabilities(host).agent ∨
     ¬Capabilities(host).waitsForCheck ∨ ¬busy)

Decision ≜
    CASE request ∈ {"inspect", "repair"} → "allow"
      □  request = "publish" →
           IF Capabilities(host).agent ∧ Guarded(SelectedChecks) ≠ {}
           THEN "deny" ELSE "allow"
      □  request = "complete" → IF Guard THEN Capabilities(host).completion ELSE "allow"

QueuesCorrection ≜ Decision = "queue" ∨ (host = "claude" ∧ Decision = "block")
EndsTurn ≜ request = "complete" ∧ (Decision ∈ {"allow", "warn", "queue"} ∨ host = "claude")

Respond ≜
    ∧ CanRespond
    ∧ inTurn′ = IF EndsTurn THEN FALSE ELSE inTurn
    ∧ correctionTurn′ = IF EndsTurn THEN FALSE ELSE correctionTurn
    ∧ correctionPending′ = (correctionPending ∨ (QueuesCorrection ∧ ¬correctionQueued))
    ∧ reply′ = [ready ↦ TRUE, decision ↦ Decision]
    ∧ warnings′ = warnings + IF Decision = "warn" THEN 1 ELSE 0
    ∧ corrections′ = corrections + IF QueuesCorrection ∧ ¬correctionQueued THEN 1 ELSE 0
    ∧ correctionQueued′ = (correctionQueued ∨ QueuesCorrection)
    ∧ wakes′ = wakes + IF QueuesCorrection ∧ ¬correctionQueued ∧
                          (host = "opencode2" ∨ ¬inTurn)
                       THEN 1 ELSE 0
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, requests, changes, session, visible,
                 contextPending, contextSent, contexts, work, outside, outsides, noticeVars⟩

Acknowledge ≜
    ∧ request ≠ "none" ∧ reply.ready
    ∧ request′ = "none"
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work,
                 correctionPending, correctionTurn, outside, outsides, noticeVars⟩
```

Section: *Checks* and result delivery

A completed check replaces the reports. Changed findings invalidate old delivery and correction deduplication. Report updates visibility and either publishes event context or retains it for *Deliver*. Recovery clears stale context.

```
BeginCheck ≜
    ∧ active ∧ ¬busy ∧ changes < MaxChanges
    ∧ busy′ = TRUE
    ∧ UNCHANGED ⟨host, active, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work,
```

$\langle correctionPending,\ correctionTurn,\ outside,\ outsides,\ noticeVars\rangle$

The result-ordering and baseline rules are specified in *Overwatch.tla*. This layer accepts a completed result at that boundary, then constrains what every host must expose and which requests it may hold behind a check.

$FinishCheck(f) \triangleq$
$\quad \wedge\ active \wedge busy \wedge changes < MaxChanges$
$\quad \wedge\ reports' = f \wedge busy' = \text{FALSE} \wedge changes' = changes + 1 \wedge outside' = \text{FALSE}$
$\quad \wedge\ \text{IF}\ f \neq reports$
$\qquad \text{THEN}\ \wedge\ contextSent' = \text{FALSE} \wedge correctionQueued' = \text{FALSE}$
$\qquad\qquad \wedge\ contextPending' = \text{FALSE}$
$\qquad \text{ELSE}\ \text{UNCHANGED}\ \langle contextSent,\ correctionQueued,\ contextPending\rangle$
$\quad \wedge\ \text{UNCHANGED}\ \langle host,\ active,\ baseline,\ inherited,\ selected,\ request,$
$\qquad\qquad reply,\ requests,\ session,\ visible,\ contexts,\ warnings,\ corrections,\ inTurn,\ wakes,\ work,$
$\qquad\qquad correctionPending,\ correctionTurn,\ outsides,\ noticeVars\rangle$

A check that finishes on another process's change, with no session edit since the last result, replaces the reports for status and guards alone: it neither releases correction deduplication nor counts as a change the session made, and *Report* gives it no failure context. A context still waiting for the old reports is dropped, as a newer result replaces it.

$OutsideFinish(f) \triangleq$
$\quad \wedge\ active \wedge busy \wedge outsides < MaxChanges \wedge Capabilities(host).noticesOutside$
$\quad \wedge\ reports' = f \wedge busy' = \text{FALSE} \wedge outside' = \text{TRUE} \wedge outsides' = outsides + 1$
$\quad \wedge\ contextPending' = (contextPending \wedge f = reports)$
$\quad \wedge\ \text{UNCHANGED}\ \langle host,\ active,\ baseline,\ inherited,\ selected,\ request,$
$\qquad\qquad reply,\ requests,\ changes,\ session,\ visible,\ contextSent,$
$\qquad\qquad correctionQueued,\ contexts,\ warnings,\ corrections,\ inTurn,\ wakes,\ work,$
$\qquad\qquad correctionPending,\ correctionTurn,\ noticeVars\rangle$

$Told \triangleq Notify \wedge \neg outside$
$Report \triangleq$
$\quad \wedge\ active \wedge \neg busy$
$\quad \wedge\ visible' = [known \mapsto \text{TRUE},\ failures \mapsto reports]$
$\quad \wedge\ contextPending' = (Told \wedge \neg contextSent \wedge Capabilities(host).delivery \in \{\text{“turn”},\ \text{“context”}\})$
$\quad \wedge\ contextSent' = (contextSent \vee (Told \wedge Capabilities(host).delivery = \text{“event”}))$
$\quad \wedge\ contexts' = contexts + \text{IF}\ Told \wedge \neg contextSent \wedge Capabilities(host).delivery = \text{“event”}\ \text{THEN}\ 1\ \text{ELSE}\ 0$
$\quad \wedge\ \text{UNCHANGED}\ \langle host,\ active,\ busy,\ reports,\ baseline,\ inherited,\ selected,$
$\qquad\qquad request,\ reply,\ requests,\ changes,\ session,$
$\qquad\qquad correctionQueued,\ warnings,\ corrections,\ inTurn,\ wakes,\ work,$
$\qquad\qquad correctionPending,\ correctionTurn,\ outside,\ outsides,\ noticeVars\rangle$

$Deliver \triangleq$
$\quad \wedge\ active \wedge contextPending$
$\quad \wedge\ wakes' = wakes + \text{IF}\ host = \text{“claude”} \wedge \neg inTurn\ \text{THEN}\ 1\ \text{ELSE}\ 0$
$\quad \wedge\ contextSent' = \text{TRUE} \wedge contextPending' = \text{FALSE} \wedge contexts' = contexts + 1$
$\quad \wedge\ \text{UNCHANGED}\ \langle host,\ active,\ busy,\ reports,\ baseline,\ inherited,\ selected,$
$\qquad\qquad request,\ reply,\ requests,\ changes,\ session,\ visible,$
$\qquad\qquad correctionQueued,\ warnings,\ corrections,\ inTurn,\ work,$
$\qquad\qquad correctionPending,\ correctionTurn,\ outside,\ outsides,\ noticeVars\rangle$

Section: Shutdown restart and new work

Stop cancels an outstanding request and clears visible and pending delivery. Restart preserves project policy, takes reports as the new baseline, and resets delivery state. A submitted correction runs in a turn of its own, which on *OpenCode* 2 is always the next; any other turn is the person's. Completed tool work permits a fresh correction, except in the correction's own turn, where inspecting and repairing are its answer.

$Stop \triangleq$
$\quad \wedge\ active$
$\quad \wedge\ inTurn' = \text{FALSE}$
$\quad \wedge\ correctionPending' = \text{FALSE} \wedge correctionTurn' = \text{FALSE}$
$\quad \wedge\ active' = \text{FALSE} \wedge busy' = \text{FALSE} \wedge request' = \text{“none”}$

```
    ∧ reply′ = [ready ↦ TRUE, decision ↦ "cancelled"]
    ∧ visible′ = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending′ = FALSE ∧ noticePending′ = FALSE
    ∧ UNCHANGED ⟨host, reports, baseline, inherited, selected, requests,
                 changes, session, contextSent, correctionQueued,
                 contexts, warnings, corrections, wakes, work, outside, outsides, noticesRaised, noticesTold⟩

Restart ≜
    ∧ ¬active ∧ session < MaxSessions
    ∧ inTurn′ = FALSE
    ∧ correctionPending′ = FALSE ∧ correctionTurn′ = FALSE
    ∧ active′ = TRUE ∧ session′ = session + 1 ∧ busy′ = FALSE
    ∧ baseline′ = reports
    ∧ request′ = "none" ∧ reply′ = NoReply ∧ requests′ = 0
    ∧ visible′ = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending′ = FALSE ∧ contextSent′ = FALSE ∧ correctionQueued′ = FALSE ∧ outside′ = FALSE
    ∧ UNCHANGED ⟨host, reports, inherited, selected, changes, contexts, warnings, corrections,
                 wakes, work, outsides, noticeVars⟩

StartTurn ≜
    ∧ active ∧ ¬inTurn ∧ inTurn′ = TRUE
    ∧ ¬(host = "opencode2" ∧ correctionPending)
    ∧ correctionTurn′ = FALSE
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued, contexts,
                 warnings, corrections, wakes, work, correctionPending, outside, outsides, noticeVars⟩

StartCorrectionTurn ≜
    ∧ active ∧ ¬inTurn ∧ correctionPending
    ∧ inTurn′ = TRUE ∧ correctionTurn′ = TRUE ∧ correctionPending′ = FALSE
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued, contexts,
                 warnings, corrections, wakes, work, outside, outsides, noticeVars⟩

WorkCompleted ≜
    ∧ active ∧ work < (MaxSessions + 1) ∗ MaxRequests
    ∧ work′ = work + IF correctionTurn THEN 0 ELSE 1
    ∧ correctionQueued′ = (correctionTurn ∧ correctionQueued)
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, contexts, warnings, corrections, inTurn, wakes,
                 correctionPending, correctionTurn, outside, outsides, noticeVars⟩
```

Section: Notices of a *TCR* revert

A host that runs *TCR* reverts an edit that adds a failure, and the agent has to be told why. A host that delivers by event appends the notice to the tool result at once. *OpenCode* 2 queues it for the next model request. A notice is not a failure: a check that finds nothing guarded clears a queued failure, never a queued notice. Each notice is delivered at most once and only a shutdown discards one that is still queued.

```
RevertNotice ≜
    ∧ active ∧ Capabilities(host).tcr ∧ noticesRaised < MaxNotices
    ∧ noticesRaised′ = noticesRaised + 1
    ∧ IF Capabilities(host).delivery = "context"
        THEN noticePending′ = TRUE ∧ UNCHANGED noticesTold
        ELSE noticesTold′ = noticesTold + 1 ∧ UNCHANGED noticePending
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
```

$\quad\quad\quad

$$\rightsquigarrow (reply.ready \vee \neg active)$$